

GraphWord

Arquitectura distribuida, demostración en AWS, pruebas CI/CD y benchmark

Guillermo Cubas Granado · Grado en Ciencia e Ingeniería de Datos · Universidad de Las Palmas de Gran Canaria · 7 de octubre de 2026

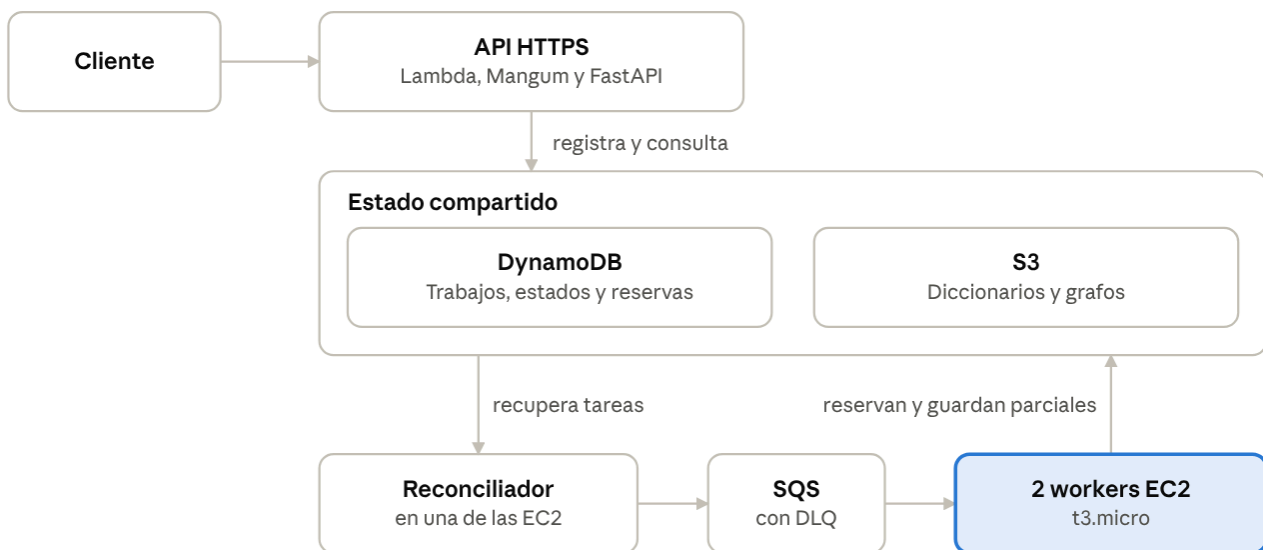
1 Objetivo y arquitectura implementada

GraphWord es un proyecto en Python que construye grafos de palabras y permite consultarlos mediante una API. Cada palabra es un nodo y se conecta con otra cuando ambas tienen la misma longitud y difieren en una sola letra. El grafo es no dirigido y no ponderado.

He separado la aplicación en API, motor de grafos, workers, reconciliador y adaptadores de almacenamiento. El motor no depende de AWS, por lo que puede probarse en local. La documentación distingue la vista de aplicación de la tecnológica, siguiendo una adaptación académica de TOGAF y conceptos de ArchiMate, sin afirmar conformidad formal.

Tecnología	Función y motivo de elección
FastAPI, Lambda y Mangum	API HTTPS con validación y Swagger. Mangum adapta FastAPI a Lambda sin rehacer las rutas.
Dos EC2 t3.micro	Workers que reparten la construcción. Una de ellas aloja también el reconciliador y la API privada de respaldo.
S3	Diccionarios, entradas, grafos parciales y finales compartidos entre procesos.
DynamoDB	Registro duradero de trabajos, estados, reservas, intentos y dependencias.
SQS y DLQ	Avisos a los workers y mensajes con recepciones fallidas; separan la petición del cálculo.
CloudFormation y SSM	Recursos como código y administración de las EC2 sin abrir SSH ni publicar el puerto 8000.
GitHub Actions y runner EC2	Pruebas y despliegue automatizado. El runner es una tercera EC2, no un worker del grafo.

La API solo registra el trabajo; el cálculo lo hacen los workers



arquitectura · 7 componentes

El flujo completo se resume así: el cliente llama por HTTPS a la API (Lambda, Mangum y FastAPI), que registra el trabajo en DynamoDB y lee o escribe en S3. El reconciliador publica los avisos en SQS, los dos workers EC2 los consumen y el resultado final se guarda en S3.

2 Datos, algoritmos y decisiones de diseño

Diccionarios y construcción distribuida

Los diccionarios combinan una fuente léxica Hunspell/SCOWL de LibreOffice y las pronunciaciones de CMUdict. Se conserva su intersección, filtrando letras ASCII minúsculas y longitudes de tres a ocho. Se eliminan duplicados y se registran fuentes, versiones, huellas y licencias. Es un filtro reproducible, no una garantía lingüística perfecta.

El catálogo y los archivos se publican en S3. El diccionario `words3` contiene 628 palabras y produce 3815 aristas. La API permite seleccionar el diccionario por ID y verificar su integridad, o recibir una lista del usuario con normalización sintáctica.

Para encontrar conexiones genero patrones: `cat` y `bat` comparten `*at`. Reparto los patrones mediante SHA256 y el número de particiones. No divido las palabras en bloques independientes, porque se perderían las conexiones entre bloques.

Con ocho particiones se crean ocho tareas y una reducción. Cada tarea lee las palabras y procesa solo sus patrones. Los dos workers recogen las tareas disponibles sin un reparto fijo de cuatro para cada uno. Al terminar, un worker une los parciales guardados en S3.

Consultas disponibles

Operación	Implementación y decisión
Camino mínimo	BFS, porque todas las aristas cuestan uno.
Todos los caminos	Exploración de caminos simples con límites de tiempo, profundidad, estados y resultados.
Camino más largo	Mejor camino simple entre dos nodos; solo se demuestra que es el máximo si <code>complete=true</code> .
Grados y aislados	Cuenta de vecinos y selección por grado, grado máximo o grado cero.
Subgrafos densos	Regiones k-core con un grado interno mínimo; no se presenta como Louvain.

Robustez, seguridad y operación

DynamoDB controla la reserva de cada trabajo; SQS solo avisa. El heartbeat renueva la reserva mientras el worker calcula, y el reconciliador recupera tareas y habilita la reducción. Una clave de idempotencia evita duplicar peticiones. Se admiten mensajes repetidos, pero no se promete ejecución exactamente una vez.

La entrada HTTPS exige contraseña y sesión firmada. Lambda guarda sal y hash, no la contraseña en texto. La Function URL admite invocación pública; la protección está en la aplicación, por lo que los rechazos también pueden consumir recursos. S3 es privado y los procesos usan los roles existentes del laboratorio, sin ampliar permisos.

Al terminar se pausa Lambda y se detienen por separado los workers y el runner. Los datos se conservan. No hay autoscaling ni garantía de alta disponibilidad: la reducción en memoria y el reconciliador único limitan la escala.

3 Demostración funcional de la API en AWS

Acceso y ejemplo de consultas

Se accede al Swagger HTTPS de GraphWord con el usuario `graphword`; la contraseña se facilita por separado. Lambda debe estar reanudada y, para los trabajos distribuidos, deben estar activos los dos workers y el reconciliador (ver el último apartado de esta sección).

En Swagger se usa *Try it out*, *Execute* y se observa la respuesta real del servidor. Primero se envía `POST /v1/graphs`:

```
{
  "words": ["cat", "bat", "bad", "dad", "mat", "rat", "zzz"],
  "partitions": 2
}
```

El resultado esperado es HTTP 201, siete nodos, ocho aristas y dos componentes. Se guarda el `graph_id` y se utiliza en las siguientes consultas, donde `...` representa `/v1/graphs/{graph_id}`:

Prueba	Entrada y resultado esperado
POST <code>.../queries/shortest-path</code>	start=cat, end=dad: cat, bat, bad, dad
POST <code>.../queries/all-simple-paths</code>	Mismos extremos: cinco caminos y <code>complete=true</code> en el ejemplo pequeño
POST <code>.../queries/longest-simple-path</code>	cat, mat, rat, bat, bad, dad; <code>complete=true</code>
GET <code>.../nodes</code>	<code>degree=0</code> devuelve zzz; <code>degree=4</code> devuelve bat
GET <code>.../dense-subgraphs</code>	<code>minimum_degree=3</code> devuelve bat, cat, mat, rat

Para las búsquedas completas del ejemplo se usan 10000 estados, profundidad 10 y un timeout de cinco segundos; `all-simple-paths` admite hasta 100 resultados.

Construcción desde S3 y evidencia real

Se consulta `GET /v1/dictionaries` y se envía `POST /v1/jobs/dictionary-builds` con `{"dictionary_id":"words3","partitions":8}`. HTTP 202 significa aceptado, no terminado. Se consulta `GET /v1/jobs/{job_id}` hasta ver `SUCCEEDED` y se obtiene `result.graph_id`. El resumen debe indicar 628 nodos, 3815 aristas y 12 componentes; el camino mínimo cat-dad es cat, cad, dad.

Estado del entorno tras la demostración

Al terminar se pausa Lambda y se detienen las EC2 para controlar el coste, así que la demostración en vivo solo funciona con el entorno activo: Lambda reanudada y los dos workers y el reconciliador en ejecución. Los datos se conservan, por lo que los diccionarios y los grafos siguen disponibles al reanudarlo.

4 Informe de pruebas automáticas y CI/CD

Integración continua

GitHub Actions instala el proyecto y ejecuta `unittest` en Python 3.11 y 3.12. Los 122 casos cubren motor, API, persistencia, coordinación, diccionarios, seguridad y despliegue. Los tests del motor incluyen una comparación con un constructor independiente de parejas de palabras.

Moto simula S3, DynamoDB y SQS para obtener pruebas repetibles sin desplegar recursos. No es AWS real ni LocalStack; los *smoke tests* de despliegue verifican AWS por separado. Que pasen todos los casos no equivale a una cobertura completa de líneas o ramas.

Entorno	Casos	Resultado	Tiempo <code>unittest</code>
Ubuntu 22.04 / Python 3.11	122	OK	11,518 s
Ubuntu 22.04 / Python 3.12	122	OK	12,843 s

Son las mismas 122 pruebas en dos intérpretes, no 244 casos distintos. Los artefactos `unittest-3.11` y `unittest-3.12` conservan los registros. El comando reproducible es:

```
python -m unittest discover -s tests -v
```

Despliegue automatizado y alcance

Tras aprobar la CI, el CD autorizado utiliza un runner EC2 con rol temporal, descarga el commit probado, publica el release y aplica CloudFormation. Comprueba la API y los workers mediante SSM y, finalmente, solicita y verifica la parada de las EC2 de aplicación. Requiere repositorio privado, rama autorizada y `ENABLE_LAB_CD=true`; no despliega *pull requests* ni *forks*.

5 Arquitectura empresarial: TOGAF y ArchiMate adaptados

5.1 Qué se ha hecho

La documentación adapta el método ADM de TOGAF al tamaño del proyecto y utiliza los conceptos de capas y relaciones de ArchiMate para ordenar las vistas. Se declara expresamente que no es una certificación formal ni un modelo ArchiMate validado con una herramienta.

5.2 Interesados

Interesado	Necesidad principal
Estudiante desarrollador/operador	Implementar, demostrar, repetir y controlar el coste
Profesor evaluador	Ver corrección, diseño, trazabilidad y evidencia
Usuario analista	Construir y consultar grafos mediante la API

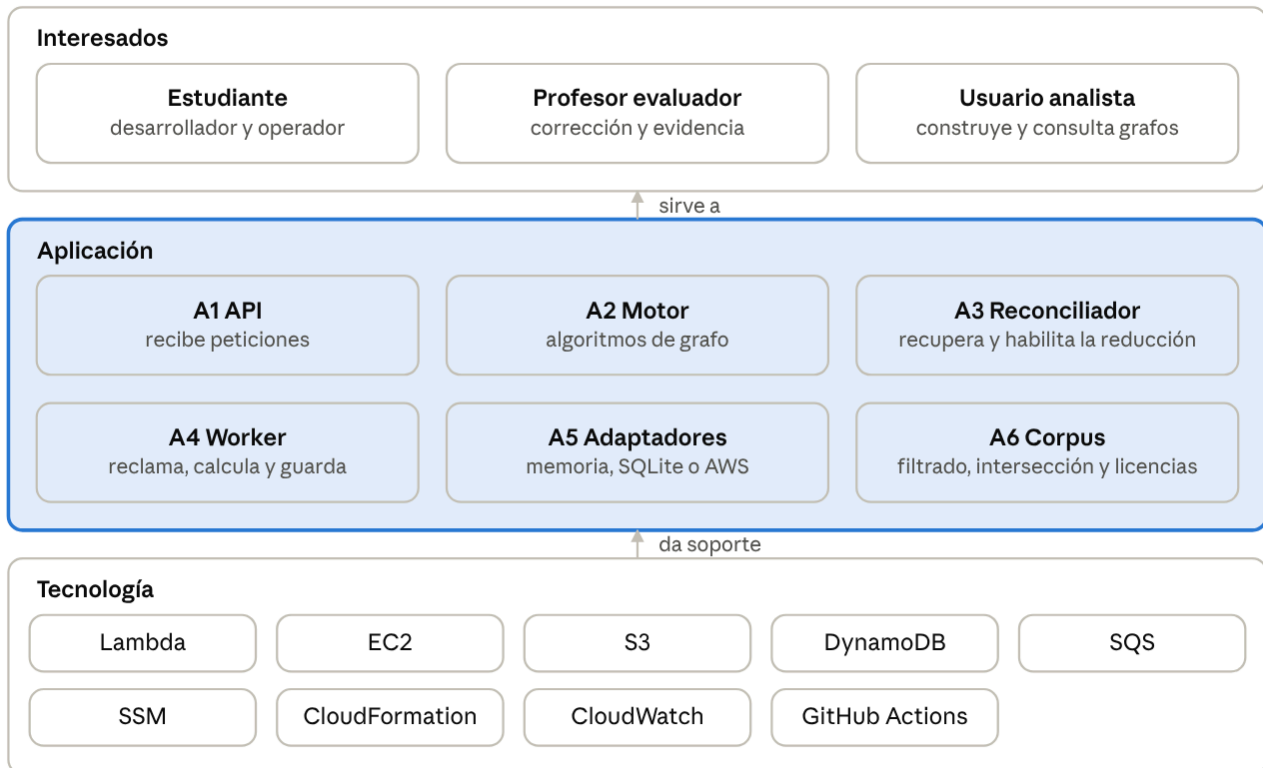
5.3 Capa de aplicación

Elemento	Responsabilidad
A1 API	Recibir construcciones y consultas
A2 Motor	Algoritmos de grafo
A3 Reconciliador	Recuperar reservas, publicar pendientes, habilitar la reducción
A4 Worker	Reclamar, renovar, calcular, guardar y confirmar
A5 Adaptadores	Persistencia y concurrencia en memoria, SQLite o AWS
A6 Preparación de corpus	Filtrado, intersección, manifiesto y licencias

5.4 Capa tecnológica

Lambda publica la API HTTPS; EC2 ejecuta los componentes; S3 conserva los datos; DynamoDB coordina los estados; SQS avisa; SSM permite la administración; CloudFormation define la infraestructura; CloudWatch conserva los registros de ejecución; GitHub Actions verifica el código.

La tecnología da soporte a la aplicación y esta sirve a los interesados



vistas por capas · 3 capas

5.5 Evolución según el ADM adaptado

La historia técnica se presenta como una transición progresiva: motor comprobable → API → persistencia local → procesos separados → particiones → servicios AWS compartidos → *hardening* → medición. Cada etapa elimina una limitación concreta sin fingir que la siguiente ya existía antes.

6 Benchmark de construcción del grafo

Metodología

Comparé uno y dos workers EC2 t3.micro en AWS. Usé diccionarios de tres a seis letras, ocho particiones y tres repeticiones por configuración: 4 diccionarios × 2 configuraciones × 3 repeticiones = 24 construcciones. Alterné el orden de uno y dos workers para reducir el efecto de las cachés.

Medí desde el envío HTTP a la API privada EC2 hasta observar el trabajo en **SUCCEEDED**. El tiempo incluye S3, DynamoDB, SQS, workers, reducción y consultas de estado.

Resultados observados

Los tiempos son medianas de tres repeticiones, en segundos. La aceleración es el tiempo con un worker dividido entre el tiempo con dos; un valor mayor que uno indica una construcción más rápida. El grado medio se calcula como $2 \cdot \text{aristas} / \text{nodos}$.

Diccionario	Nodos	Aristas	Grado medio	1 worker (s)	2 workers (s)	Aceleración
words3	628	3815	12,15	1,966	1,417	1,39 veces
words4	1889	7153	7,57	1,987	1,424	1,40 veces
words5	2677	3171	2,37	2,158	1,481	1,46 veces
words6	3676	2332	1,27	2,084	1,498	1,39 veces

Con dos workers, la mediana del tiempo se redujo entre un 28 % y un 31 %. Los 24 grafos coincidieron con el constructor de referencia.

Solo conservo la mediana de cada configuración, sin mínimos ni máximos, por lo que no puedo cuantificar la dispersión entre repeticiones. Con tres repeticiones por configuración, las diferencias de unas pocas décimas de segundo entre diccionarios deben leerse con prudencia.

Interpretación

Por qué la aceleración es de 1,4 y no de 2. Los tiempos casi no crecen con el tamaño del diccionario: con un worker pasan de 1,97 s a 2,08 s aunque los nodos se multiplican por casi seis. Esto indica que domina un coste fijo (envío de la petición, lectura y escritura en S3, coordinación con DynamoDB y SQS, reducción final y consultas de estado) que no se reparte entre workers. Es el comportamiento que describe la ley de Amdahl.

Como estimación, si se supone que solo el tiempo restante se divide a la mitad con el segundo worker, la parte paralelizable y la fija serían aproximadamente:

Diccionario	Parte paralelizable (s)	Parte fija (s)
words3	1,10	0,87
words4	1,13	0,86
words5	1,35	0,80
words6	1,17	0,91

La parte fija se mantiene entre 0,8 y 0,9 s en los cuatro diccionarios, es decir, alrededor del 40 % del tiempo con un worker. Es un modelo simple construido a partir de medianas, no una medición directa de cada fase, así que debe leerse como orden de magnitud. Con un

coste fijo de ese tamaño, aunque se añadieran más workers la mejora máxima quedaría limitada a unas 2,3 veces, y con ocho particiones el reparto entre más de dos workers tampoco sería equilibrado.

Por qué más palabras no implican más aristas. Aumentar la longitud de las palabras no aumenta las aristas: en este corpus, words4 tiene más conexiones que words6 pese a tener la mitad de nodos. La explicación probable es que, a mayor longitud, el diccionario cubre una fracción cada vez menor de las palabras posibles, por lo que es más raro que dos palabras reales difieran en una sola letra. Lo refleja el grado medio, que baja de 12,15 en words3 a 1,27 en words6. Es una explicación coherente con los datos, pero no la he contrastado con un análisis específico del corpus.

7 Conclusiones, limitaciones y trabajo futuro

Conclusiones

- GraphWord construye y consulta grafos de palabras de forma correcta: los resultados coinciden con un constructor independiente en las 24 construcciones del benchmark y en los 122 casos de prueba.
- La arquitectura con Lambda, EC2, S3, DynamoDB y SQS permite repartir la construcción entre workers. Con dos t3.micro la mediana del tiempo baja entre un 28 % y un 31 %.
- La CI/CD automatiza las pruebas en dos versiones de Python y el despliegue, y apaga los recursos al terminar para controlar el coste.